Instructor Handout 9: Priority, Round Robin, Multilevel Queues, and Multiprocessor Scheduling

Lecture 9 — closes Unit I

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Instructor Copy — Not for Student Distribution

Concepts That Click

9.1 Before You Teach This Lecture

- **Silberschatz, Galvin & Gagne, *Operating System Concepts*, 8e** — Ch. 5 §5.3: Priority (p. 192), Round Robin (p. 194), Multilevel Queue (p. 196, Fig. 5.6), Multilevel Feedback Queue (p. 197); §5.5 multiple-processor scheduling (p. 200 onward). Primary anchor; *page-verified* via the course index. The subsection-level detail of §5.5 (affinity, load balancing, multicore) is indexed only at section level, so treat it as the section's general treatment.
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e** — Ch. 9 §9.2 (scheduling algorithms, p. 410) for RR, priority and feedback scheduling; Ch. 10 §10.1 (multiprocessor scheduling, p. 453). *Page-verified.*
- **Bovet & Cesati, *Understanding the Linux Kernel*, 3e** — Ch. 7 (pp. 258–293): priorities, timeslices, per-CPU runqueues and SMP runqueue balancing. Use for "what Linux actually does". *Page-verified.*
- **Know the running workload W cold** (Table 9.1). Every slide reuses it; students will copy your Gantt charts, so practise each one once on paper before class.
- **State the conventions before the first chart:** lower number = higher priority; ties to the earlier arrival; in RR an arrival at a quantum boundary joins the queue *before* the preempted process. Most student disagreements about RR answers are disagreements about this tie rule.
- **Prerequisite check (two minutes):** TAT, WT, RT and the FCFS/SJF/SRTF Gantt method from Lecture 8; the timer interrupt from Lecture 2; the context switch from Lecture 6.

Teaching note: The thesis to keep on the board all hour: *no policy wins every column.* Leave a blank five-row comparison table (FCFS, NP priority, P priority, RR $q = 3$, MLFQ × avg WT, avg TAT, avg RT, P5 done at, switches) in a corner and fill one row after each section. The finished table is the lecture's conclusion.

Process	Arrival	τ	Priority	Character
P1	0	8	3	medium batch job
P2	1	4	1	interactive, important
P3	2	9	4	long compute job
P4	3	5	2	system daemon
P5	4	2	5	short, low-importance

Table 9.1: Running workload W (ms). Total work 28 ms.

9.2 Opening: Where FCFS and SJF Left Us

A laptop runs a video call, a compiler and a backup. FCFS lets the backup block the call (the **convoy effect**); SJF needs burst lengths nobody knows. Real schedulers combine four ideas — **Priority**, **RR**, **MLQ**, **MLFQ** — and must also decide which CPU a process runs on.

Recall the metrics: TAT = completion − arrival; WT = TAT − τ; RT = first dispatch − arrival. FCFS on W runs P1 0–8, P2 8–12, P3 12–21, P4 21–26, P5 26–28: WT $= 0, 7, 10, 18, 22$, average $57/5 = 11.4$ ms; TAT average 17.0 ms. P5 needs 2 ms and waits 22.



Figure 9.1: FCFS baseline on W.

Board: the baseline. *On the board, in order:* (i) a time axis 0–28 with ticks every 2 ms; (ii) gold arrival arrows under 0,1,2,3,4 labelled P1–P5; (iii) the five FCFS bars; (iv) the WT column computed aloud, P5 last. *Say:* "Twenty-two milliseconds of waiting for two of work. Today's question is who should have gone first."

> **Teaching note:** Ask students to draw the axis and arrows themselves before you draw bars; they reuse that axis for four more charts. Arrivals belong *on the axis*, never inside bars — a habit worth enforcing now.

9.3 Priority Scheduling

Definition (Priority scheduling). *Allocate the CPU to the ready process with the highest priority; equal priorities are broken FCFS.*

Priorities are internal (time limits, memory needs, I/O-to-CPU burst ratio) or external (importance, payment, department). SJF is the special case where priority is the predicted next burst. Non-preemptive priority decides only when the CPU frees; preemptive priority reacts to each arrival.

Non-preemptive on W: P1 0–8 (alone at 0); at 8, ready = {P2(1), P3(4), P4(2), P5(5)}, so P2 8–12, P4 12–17, P3 17–26, P5 26–28. WT $= 0, 7, 15, 9, 22$, average 10.6; TAT average 16.2.

Preemptive on W: P1 0–1; P2 arrives at 1 and preempts (P1 keeps 7); P2 1–5; at 5 P4 (2) beats P1 (3): P4 5–10; P1 10–17; P3 17–26; P5 26–28. WT $= 9, 0, 15, 2, 22$, average 9.6; TAT average 15.2; RT average 7.8.

Concepts That Click

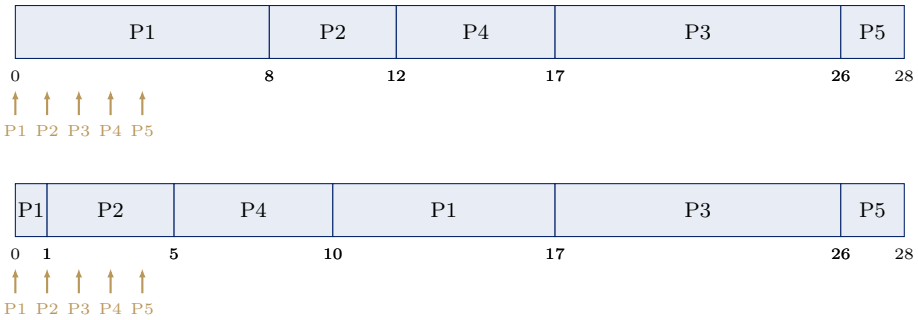


Figure 9.2: Non-preemptive (top) and preemptive (bottom) priority on W.

Board: preemption at $t = 1$. *On the board, in order:* (i) reuse the axis; (ii) draw P1's bar from 0 and stop the chalk at 1; (iii) write "P2 (1) > P1 (3)" above the cut; (iv) finish the chart, writing P1's remaining work (7) above its resumed bar. *Say:* "The important process waits zero. Who paid?" — answer: P1 (9 ms) and, as always, P5 (22).

Board: a second instance, all arrivals at 0 (new numbers): P1 (4, pr 2), P2 (3, pr 1), P3 (6, pr 3), P4 (2, pr 2). *On the board, in order:* list the four rows; sort by priority, breaking the P1/P4 tie by arrival order (P1 listed first); draw P2 0–3, P1 3–7, P4 7–9, P3 9–15. WT $= 3, 0, 9, 7$; average $19/4 = 4.75$ ms. *Live hook:* ask "with every arrival at 0, does preemption change anything?" (No — no arrival can interrupt.)

Starvation and aging. **Starvation**: a ready process that never runs because better work keeps arriving. **Aging**: raise a process's priority as it waits. With priorities 0–63 and aging of 1 step per 10 ms, the worst case (entering at 63) reaches 0 after $63 \times 10 = 630$ ms; nothing then outranks it, so its wait is bounded.

> **Teaching note:** Where students get stuck: the direction of the priority scale. Some textbooks and exam papers use "larger is higher". Make them write the convention at the top of every answer. Socratic question worth asking live: "P1 started first but finished after P2 and P4 — unfair? What would SRTF do at $t = 1$?" SRTF makes the same preemption (P2's 4 ms < P1's remaining 7) for a different reason: SJF is a priority algorithm with a different key.

9.4 Round Robin

Definition (Round Robin). *FCFS with timer preemption: each process runs for at most one quantum q, then goes to the tail of the ready queue.*

RR, $q = 3$, on W (track the queue at each dispatch): $t = 3$: [P2, P3, P4] then P1 rejoins → [P2, P3, P4, P1]; $t = 6$: P5 (arrived 4) precedes P2 → [P3, P4, P1, P5, P2]; $t = 15$: [P5, P2, P3, P4, P1], and P5 (2) and P2 (1 left) finish without a full quantum; $t = 21$: [P4, P1, P3] drains to 23, 25, 28. First dispatch 0,3,6,9,15; completion 25,18,28,23,17; TAT average 20.2; WT $= 17, 13, 17, 15, 11$, average 14.6; RT $= 0, 2, 4, 6, 11$, average 4.6.

Board: the RR queue as a column. *On the board, in order:* (i) draw the axis; (ii) to the right, a vertical "ready queue" column; (iii) at each quantum boundary, first add arrivals to the queue, *then* append the preempted process, then pop the head and draw its bar. *Say at $t = 6$:* "P5 arrived at 4, while P2 was running — so P5 is ahead of P2."

Board: a second instance (new numbers, all at 0): P1 5, P2 3, P3 4. $q = 2$: P1 0–2, P2 2–4, P3 4–6, P1 6–8, P2 8–9, P3 9–11, P1 11–12; WT $= 7, 6, 7$, average $20/3 \approx 6.67$. $q = 4$: P1 0–4, P2

Concepts That Click

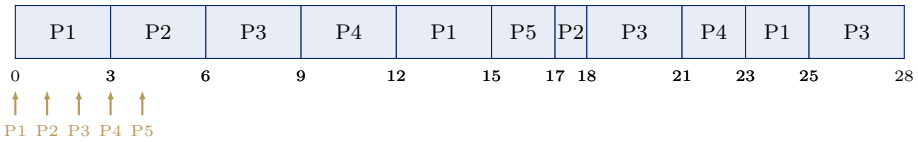


Figure 9.3: RR, $q = 3$, on W.

4–7, P3 7–11, P1 11–12; WT $= 7, 4, 7$, average 6.0. *Live hook:* "predict which quantum gives the lower average WT before I draw it."

The quantum sweep on W (context switches = changes of running process): $q = 1$: WT 13.0, TAT 18.6, RT 1.2, 25 switches; $q = 2$: 13.0, 18.6, 2.8, 13; $q = 3$: 14.6, 20.2, 4.6, 10; $q = 4$: 13.0, 18.6, 6.0, 8; $q = 5$: 12.6, 18.2, 7.4, 6; $q \geq 9$: 11.4, 17.0, 11.4, 4 (RR = FCFS). RT grows with q; WT does not move monotonically: with $q = 4$, P2's 4 ms burst finishes in one slice; with $q = 3$ it needs two and waits a whole round.

Overhead. With switch cost s, the useful fraction is $q/(q+s)$; with $s = 0.5$: $q = 1$ gives 66.7%, $q = 3$ 85.7%, $q = 10$ 95.2%. On W, $q = 1$'s 25 switches add 12.5 ms to a 28 ms schedule; $q = 3$'s 10 add 5 ms. Rule of thumb: q large compared with s, yet small enough that about 80% of bursts finish within one quantum.

Bound. With n ready processes, no process waits more than $(n-1)q$ for its next quantum. On W, $q = 3$: $4 \times 3 = 12$; the largest RT is P5's 11.

> **Teaching note:** Where students get stuck: the tie rule, and counting a finished process as "preempted". A process whose burst ends inside its quantum leaves voluntarily and never rejoins. Socratic question: "$q = 2$ and $q = 4$ give 13.0 but $q = 3$ gives 14.6 — does a bigger quantum always reduce waiting?" Let them argue before revealing the P2 explanation.

9.5 Multilevel Queue Scheduling

Definition (Multilevel queue). *The ready queue is split into classes; each process is permanently assigned to one queue by type, and each queue has its own algorithm.*

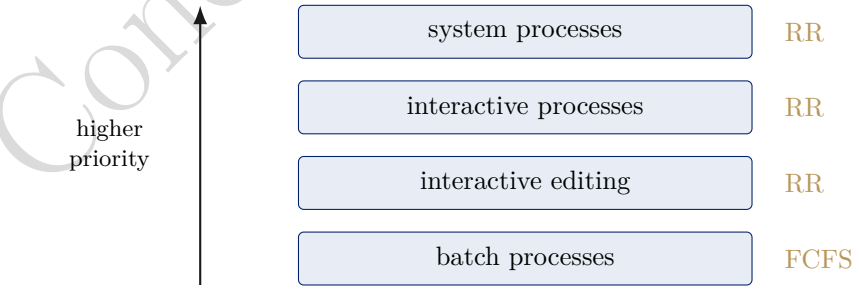


Figure 9.4: Multilevel queue — after Silberschatz et al., Ch. 5 §5.3, p. 196 (Fig. 5.6).

How to present it: draw the four boxes top to bottom first, then the policy labels on the right, and the "higher priority" arrow last — the arrow is the policy *between* queues, which is the point of the next sentence. Between-queue policy is either fixed preemptive priority (lower queues can starve) or time slicing (e.g. 80% foreground RR, 20% background FCFS; nothing starves). MLQ is cheap (class chosen once) and inflexible (a process cannot change class).

Concepts That Click

Board: the deck's two-queue example. Foreground RR $q = 2$, background FCFS, fixed preemptive priority; A (0, 4, fg), B (1, 6, bg), D (2, 3, bg), C (6, 3, fg). A 0–4; B 4–6; C arrives at 6 and preempts B (4 left); C 6–9; B 9–13; D 13–16. WT: A 0, C 0, B 6, D 11; average 4.25.

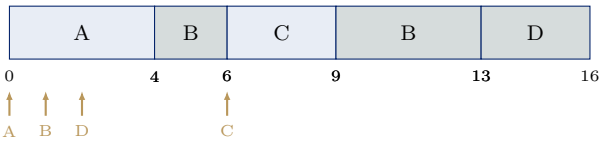


Figure 9.5: Two-queue MLQ (foreground light, background shaded).

Board: a second instance (new numbers): A (0, 3, fg), B (0, 5, bg), C (2, 4, fg), same policies. A 0–2; at 2 A's quantum expires and C has just arrived, so C goes first: C 2–4, A 4–5, C 5–7; only then B 7–12. WT = 2 (A), 1 (C), 7 (B). *Live hook:* "B arrived at time 0 — when does it first run?" (At 7, after the foreground drains.)

> **Teaching note:** Where students get stuck: assuming a preempted background process goes to the *tail* of its queue. In fixed-priority MLQ with FCFS in the background, it resumes at the head — B resumes at 9 before D.

9.6 Multilevel Feedback Queue

Definition (Multilevel feedback queue). *Processes move between queues: using a full quantum demotes; waiting too long may promote (aging).*

Five parameters define an MLFQ: number of queues; algorithm per queue; demotion rule; promotion rule; entry queue. It can mimic RR, FCFS, or an SJF approximation without knowing any burst.

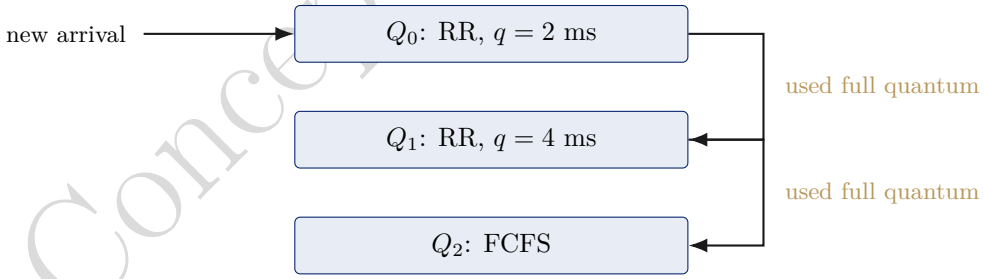


Figure 9.6: Three-level MLFQ — after Silberschatz et al., Ch. 5 §5.3, p. 197.

How to present it (coordinate-map order): the three boxes top to bottom; the "new arrival" arrow into Q_0; then each demotion arrow, saying "used full quantum" as you draw it. Draw no upward arrow yet — introduce promotion only when starvation comes up.

Board: MLFQ on W. 0–10: each process gets 2 ms in Q_0; only P5 finishes (at 10). 10–23: Q_1 runs P1 (10–14, demoted), P2 (14–16, done), P3 (16–20, demoted), P4 (20–23, done). 23–28: Q_2 runs P1 (to 25), P3 (to 28). WT = $17, 11, 17, 15, 4$, average 12.8; TAT 18.4; RT = $0, 1, 2, 3, 4$, average 2.0; 10 switches.

Board: the late short arrival. P1 (0, 9), P2 (1, 3), P3 (2, 6), P4 (8, 2). 0–6: three Q_0 slices, all demoted; 6–8: P1 in Q_1; at 8 P4 enters Q_0 and preempts P1 (to the tail of Q_1); P4 8–10 (WT 0);

Concepts That Click

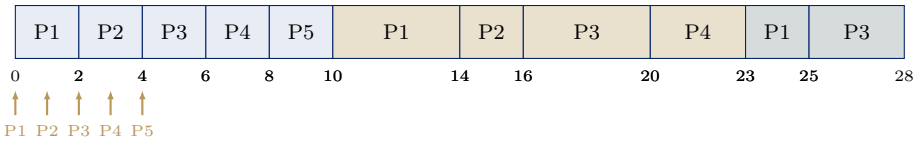


Figure 9.7: MLFQ on W (Q_0 light, Q_1 gold, Q_2 grey).

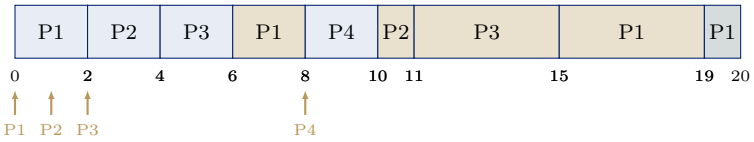


Figure 9.8: MLFQ with a late short arrival.

P2 10–11; P3 11–15; P1 15–19 in Q_1 and 19–20 in Q_2. WT $= 11, 7, 7, 0$, average 6.25; TAT average 11.25. *Live hook:* stop at $t = 8$ and ask "draw the next bar".

> **Teaching note:** Where students get stuck: what a preempted Q_1 process gets when it resumes — a fresh quantum or the remainder. Tell them the deck's examples give the same completion times either way, and that an exam must state its rule. Socratic question: "A CPU-bound program issues a tiny I/O just before every quantum expires — what happens?" It never uses a full quantum, so it is never demoted; the defence is cumulative per-level CPU accounting plus periodic promotion.

9.7 Multiprocessor Scheduling

Asymmetric multiprocessing: one master CPU makes all scheduling decisions and does I/O and kernel work. SMP: every CPU is self-scheduling, so scheduler data needs protection. Modern general-purpose OSes are SMP, which raises the question of one common queue versus per-CPU queues.

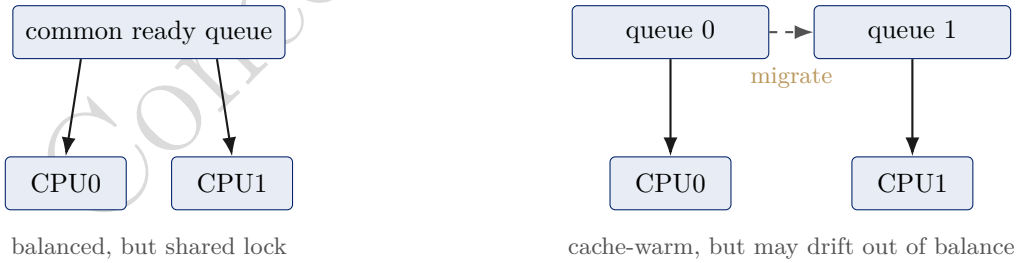


Figure 9.9: Common ready queue versus per-CPU queues.

How to present it: draw the left half, ask "what is the cost?" (one lock), then draw the right half and ask the same question (imbalance); add the dashed "migrate" arrow last — it introduces load balancing.

Affinity: a process prefers its last CPU because the cache holds its data; migration costs a cache refill. Soft affinity (OS tries) versus hard affinity (process restricts itself, e.g. `sched_setaffinity`); on NUMA, affinity also keeps the process near its memory. **Load balancing:** push migration (a periodic task moves work off busy CPUs) and pull migration (an idle CPU takes work); Linux uses

Concepts That Click

both with per-CPU runqueues and periodic balancing. Migrate only when the imbalance outweighs the cache cost. **Multicore:** a memory stall idles a core after a cache miss; hardware threads let the core switch threads; the OS schedules software threads onto logical CPUs and the hardware schedules hardware threads onto the pipeline.

Board: the deck's balance example. Bursts 6, 5, 4, 3, 2, 2 on two CPUs. Static split (J1–J3 / J4–J6): finish 15, CPU1 idle 8. Common queue: CPU0 J1 0–6, J4 6–9, J5 9–11; CPU1 J2 0–5, J3 5–9, J6 9–11: finish $11 = 22/2$.

Board: a second instance (new numbers): bursts 9, 4, 4, 3, 3, 1 (total 24). Static split: $9 + 4 + 4 = 17$ vs. $3 + 3 + 1 = 7$, finish 17. Common queue: CPU0 J1 0–9, J5 9–12; CPU1 J2 0–4, J3 4–8, J4 8–11, J6 11–12: finish $12 = 24/2$. *Live hook:* "what is the best any schedule could do?" (Total/CPUs.)

> **Teaching note:** Where students get stuck: thinking more CPUs automatically means proportionally faster. The two examples show that balance, not CPU count, sets the finish time.

9.8 Exit Ticket / Comprehension Checks

1. What does RR become when $q \geq$ the longest burst? (FCFS.)
2. Which policy on W gives the lowest average WT, and which the lowest average RT? (Preemptive priority 9.6; MLFQ 2.0.)
3. One sentence: how does MLFQ differ from MLQ? (Processes move between queues by observed behaviour.)
4. Ten processes, $q = 20$ ms: maximum wait for the next quantum? (180 ms.)

9.9 Summary: One-Page Refresh

Policy on W	avg WT	avg TAT	avg RT	P5 done	switches
FCFS	11.4	17.0	11.4	28	4
Priority (NP)	10.6	16.2	10.6	28	4
Priority (P)	9.6	15.2	7.8	28	5
RR, $q = 3$	14.6	20.2	4.6	17	10
MLFQ (2, 4, FCFS)	12.8	18.4	2.0	10	10

Priority runs the most important ready process; aging prevents starvation. RR bounds the wait for the next quantum by $(n - 1)q$; useful fraction $q/(q + s)$; $q \geq$ longest burst gives FCFS. MLQ: fixed classes, own policies, fixed-priority or time-sliced between queues. MLFQ: demote on full quantum, promote on long wait, five parameters. Multiprocessor: SMP, common vs. per-CPU queues, affinity, push/pull balancing, hardware threads. Misconceptions to pre-empt: priority-scale direction, monotone WT in q, "RR always beats FCFS", "MLQ moves processes", "more CPUs always help".

Concepts That Click

9.10 Readings to Prepare Next Time

Lecture 10 opens Unit II: race conditions and the critical-section problem. Read Silberschatz et al., Ch. 6 §6.1–6.2 (pp. 225–229) and Stallings, Ch. 5 §5.1. Bridge sentence for the end of class: "A scheduler can stop a process at any timer interrupt. Next time: what goes wrong when two such processes share data."

Concepts That Click